

Modül 19

Internationalization

Çoklu Dil Desteği — i18n

Modül:	19 / 30
Ders Sayısı:	6 ders
Seviye:	Orta
Versiyon:	Angular v21

Hazırlayan: Claude • Türkçe Angular v21 Eğitimi

Modül 19'a Hoş Geldin

Uluslararası uygulama yapmak — birçok dilde, birçok kültürde çalışan. Angular'ın iki yaklaşımı var: built-in @angular/localize (compile-time) ve ngx-translate (runtime). Her ikisini de işliyoruz.

Öğrenecekler:

- ✓ @angular/localize ile compile-time i18n
- ✓ ngx-translate ile runtime i18n
- ✓ Çevirilerde plural ve gender
- ✓ Date, number, currency locale
- ✓ RTL (right-to-left) support
- ✓ Hangi yaklaşımı ne zaman seç

@angular/localize Setup

Compile-time i18n — Angular'ın resmi yaklaşımı.

⚙ Kurulum

```
ng add @angular/localize
```

📄 i18n Attribute

```
<!-- Template'de i18n attribute -->
<h1 i18n>Hoş geldiniz</h1>

<!-- Meaning ve description ile -->
<h1 i18n="header.welcome|Welcome message@@welcomeHeader">
  Hoş geldiniz
</h1>

<!-- Attribute i18n -->


<!-- Plural -->
<span i18n>{count, plural,
  =0 {Hiç item yok}
  =1 {Bir item}
  other {{{count}} item}
}</span>
```

📄 Extract

```
# Çevrilecek string'leri çıkar
ng extract-i18n --output-path src/locale

# messages.xlf oluşur (XLIFF format)
# Bunu çevirmenlere gönder
```

📄 Multi-Locale Build

```
// angular.json
{
  "projects": {
    "my-app": {
      "i18n": {
        "sourceLocale": "tr",
        "locales": {
          "en": "src/locale/messages.en.xlf",
          "de": "src/locale/messages.de.xlf"
        }
      },
      "architect": {
        "build": {
          "options": {
            "localize": ["tr", "en", "de"]
          }
        }
      }
    }
  }
}
```

□ Build

```
# Her locale için ayrı build
ng build --localize
```

```
# dist/my-app/tr/, dist/my-app/en/, dist/my-app/de/ klasörleri oluşur
```

ngx-translate — Runtime

Runtime'da dil değiştirme — daha esnek.

⚙ Setup

```
npm install @ngx-translate/core @ngx-translate/http-loader

// app.config.ts
import { provideTranslateService, TranslateLoader } from '@ngx-translate/core';
import { TranslateHttpLoader } from '@ngx-translate/http-loader';
import { provideHttpClient, HttpClient } from '@angular/common/http';

export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(),
    provideTranslateService({
      defaultLanguage: 'tr',
      loader: {
        provide: TranslateLoader,
        useFactory: (http: HttpClient) =>
          new TranslateHttpLoader(http, './assets/i18n/', '.json'),
        deps: [HttpClient]
      }
    })
  ]
};
```

📄 JSON Dosyaları

```
// assets/i18n/tr.json
{
  "welcome": "Hoş geldiniz",
  "buttons": {
    "save": "Kaydet",
    "cancel": "İptal"
  },
  "items_count": "{{count}} item",
  "user_greeting": "Merhaba {{name}}!"
}

// assets/i18n/en.json
{
  "welcome": "Welcome",
  "buttons": {
    "save": "Save",
    "cancel": "Cancel"
  },
  "items_count": "{{count}} items",
  "user_greeting": "Hello {{name}}!"
}
```

□ Component Kullanımı

```
import { TranslateService, TranslatePipe } from '@ngx-translate/core';

@Component({
  imports: [TranslatePipe],
  template: `
    <h1>{{ 'welcome' | translate }}</h1>
    <button>{{ 'buttons.save' | translate }}</button>
    <p>{{ 'user_greeting' | translate:{ name: userName() } }}</p>
  `
})
export class WelcomeComponent {
  private translate = inject(TranslateService);
  userName = signal('Ahmet');

  switchLang(lang: string) {
    this.translate.use(lang);
  }
}
```

□ Programmatic Translation

```
// Component'te
const text = this.translate.instant('welcome'); // Sync
this.translate.get('welcome').subscribe(text => console.log(text)); // Async

// Signal ile
welcomeText = toSignal(this.translate.get('welcome'), { initialValue: '' });
```

@angular/localize vs ngx-translate

Hangisini ne zaman seçmeli?

⚖ Detaylı Karşılaştırma

Konu	@angular/localize	ngx-translate
Tip	Compile-time	Runtime
Build	Her locale ayrı	Tek build, tüm locale
Bundle size	Daha küçük (locale başına)	Tüm dil yüklü
Performance	Daha hızlı	Runtime overhead
Dil değiştirme	Page reload	Anlık
SEO	Native (ayrı URL)	Manuel kurulum
Karmaşıklık	Build setup	Daha basit

□ Hangisi Ne Zaman?

- ✓ @angular/localize: SEO kritik, public-facing site
- ✓ @angular/localize: Performance kritik (bundle size)
- ✓ ngx-translate: Dil değiştirme runtime'da kullanıcı seçimi
- ✓ ngx-translate: Backend'den çeviri çekmek
- ✓ ngx-translate: Hızlı prototyping

Plural & Gender (ICU Format)

Türkçe'de pek yok ama İngilizce/Almanca'da kritik: tekil/çoğul.

Plural (ICU)

```
<!-- @angular/localize -->
<span i18n>{count, plural,
  =0 {Hiç item yok}
  =1 {Bir item}
  =2 {İki item}
  other {{{count}} item}
}</span>

<!-- ngx-translate (mesh-translate ile) -->
{
  "items": "{count, plural, =0{Hiç item yok} =1{Bir item} other{# item}}"
}

<!-- Template -->
{{ 'items' | translate:{ count: itemCount() } }}
```

Gender (Select)

```
<span i18n>{gender, select,
  male {Erkek kullanıcı}
  female {Kadın kullanıcı}
  other {Kullanıcı}
}</span>
```

Compound

```
<span i18n>
  {gender, select,
    male {{{name}} kendi}
    female {{{name}} kendi}
    other {{{name}}}}
  {count, plural,
    =1 {1 yorum}
    other {{{count}} yorum}
  } yaptı
</span>
```


Date, Number, Currency Locale

Built-in pipe'larla locale-aware formatting.

□ Locale Setup

```
import { registerLocaleData } from '@angular/common';
import localeTr from '@angular/common/locales/tr';
import localeEn from '@angular/common/locales/en';
import localeDe from '@angular/common/locales/de';

registerLocaleData(localeTr);
registerLocaleData(localeEn);
registerLocaleData(localeDe);

// app.config.ts
providers: [
  { provide: LOCALE_ID, useValue: 'tr' }, // Default
]
```

□ Locale-Aware Pipes

```
<!-- Date -->
{{ today | date:'long' }}
<!-- TR: 19 Mayıs 2026 15:35:00 GMT+3 -->
<!-- EN: May 19, 2026 at 3:35:00 PM GMT+3 -->
<!-- DE: 19. Mai 2026 um 15:35:00 GMT+3 -->

<!-- Number -->
{{ 1234.56 | number }}
<!-- TR: 1.234,56 -->
<!-- EN: 1,234.56 -->
<!-- DE: 1.234,56 -->

<!-- Currency -->
{{ 99.99 | currency:'TRY' }}
<!-- TR: ₺99,99 -->
<!-- EN: TRY 99.99 -->
```

□ Dynamic Locale

```
<!-- Programatik -->
<p>{{ today | date:'long':'':locale() }}</p>

@Component({...})
export class MyComponent {
  locale = signal('tr');

  switchLocale(loc: string) {
    this.locale.set(loc);
  }
}
```

RTL Support (Arapça, İbranice)

Sağdan sola yazılan diller için CSS.

↔ HTML dir Attribute

```
<html lang="ar" dir="rtl">
  <body>
    <app-root></app-root>
  </body>
</html>

<!-- Veya dinamik -->
<html [lang]="lang()" [dir]="dir()">
```

□ CSS Logical Properties

```
/* □ Eski - fiziksel */
.card {
  margin-left: 16px; /* RTL'de yanlış taraf */
  padding-right: 12px;
}

/* ✓ Modern - logical */
.card {
  margin-inline-start: 16px; /* RTL'de otomatik flip */
  padding-inline-end: 12px;
}

/* Margin/padding inline */
margin-inline-start → ltr: left, rtl: right
margin-inline-end   → ltr: right, rtl: left
padding-block-start → top
padding-block-end   → bottom

/* Border */
border-inline-start: 1px solid;
border-inline-end: 1px solid;

/* Position */
inset-inline-start: 0; /* left in LTR, right in RTL */
```

□ Direction-Aware Components

```
import { Directionality } from '@angular/cdk/bidi';

@Component({...})
export class MyComponent {
  private dir = inject(Directionality);

  isRtl = computed(() => this.dir.value === 'rtl');
}
```

Modül 19 Özeti

i18n artık bilinen bir konu. İki yaklaşımı da bilirsin, RTL'ye hazırsın, date/number locale-aware formatlayabiliyorsun.

Neler Öğrendin?

- ✓ @angular/localize ile compile-time i18n
- ✓ ngx-translate ile runtime i18n
- ✓ İki yaklaşım arasında karar verebilme
- ✓ Plural ve gender (ICU format)
- ✓ Locale-aware date, number, currency
- ✓ RTL support — CSS logical properties

Sıradaki Modül

Modül 20 — Server-Side Rendering (SSR). Performance, SEO, hydration.